



De l'énoncé au code

4PYTH – Correction guidée du défi 3

Chris CHEVALIER



Étape 1 – Comprendre l'énoncé

Question	Réponse
Entrée	<code>length</code> , la longueur voulue (un entier, 16 par défaut)
Sortie	une chaîne de caractères aléatoire de <code>length</code> caractères
Contraintes	au moins une majuscule, une minuscule, un chiffre et un caractère spécial, à des positions variables
Cas d'erreur	<code>length</code> inférieur à 12 : lever une <code>ValueError</code>
Vérification	1 000 mots de passe générés, tous validés par <code>check_password()</code> du défi 1



Étape 1 – Les outils à disposition

```
Terminal
>>> import secrets
>>> import string
>>> string.ascii_uppercase
'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
>>> string.digits
'0123456789'
>>> secrets.choice("ABC")
'B'
>>> chars = ["a", "B", "7"]
>>> secrets.SystemRandom().shuffle(chars)
>>> "".join(chars)
'7aB'
```

- `string` fournit les alphabets tout faits
- `secrets.choice()` tire **un** caractère au hasard, de façon imprévisible
- `shuffle()` mélange une **liste** sur place. Une chaîne ne se modifie pas, on travaille donc avec une liste
- `"".join(liste)` recolle la liste en une chaîne

Jamais `random` pour un secret. Il est **prévisible**. Qui connaît son état devine la suite.
`secrets` est fait pour ça.



Étape 2 – Le plan en français

Pseudo-code

```
si la longueur est inférieure à 12 : lever une ValueError

préparer les 4 catégories : majuscules, minuscules, chiffres, spéciaux
créer une liste vide de caractères
pour chaque catégorie : tirer un caractère de cette catégorie
pour chaque caractère manquant : tirer un caractère parmi toutes les catégories
mélanger la liste
recoller la liste en une chaîne et la renvoyer
```

Tirer d'abord un caractère de chaque catégorie **garantit** les 4 règles. Le mélange final évite qu'elles soient toujours aux mêmes positions.



Étape 3a – Refuser une longueur trop courte

```
challenge3.py

import secrets
import string

from challenge1 import check_password, MIN_LENGTH

def generate_password(length=16):
    """Génère un mot de passe aléatoire robuste."""
    if length < MIN_LENGTH:
        raise ValueError(f"longueur minimale : {MIN_LENGTH}")
    return ""
```

- On **réutilise** la constante `MIN_LENGTH` du défi 1. La règle des 12 caractères n'est écrite qu'à un endroit
- `raise` **arrête** la fonction et signale l'erreur à celui qui l'appelle
- Le `return ""` provisoire permet déjà d'exécuter et de tester



Étape 3b – Un caractère de chaque catégorie

```
challenge3.py

SPECIALS = "!@#$%^&*()-_+=[]{};:,.?/"

def generate_password(length=16):
    ... # vérification de la longueur
    categories = [string.ascii_uppercase,
                  string.ascii_lowercase,
                  string.digits, SPECIALS]
    chars = []
    for category in categories:
        chars.append(secrets.choice(category))
    print(chars)      # ['K', 'q', '4', '#']
    return ""
```

- Les caractères spéciaux sont **choisis par nous** et rangés dans une constante
- Une **liste de catégories** permet une boucle au lieu de quatre lignes presque identiques
- Le `print()` provisoire montre que l'étape fonctionne



Étape 3c – Compléter, mélanger, recoller

```
challenge3.py (suite)

all_chars = "".join(categories)
for _ in range(length - len(chars)):
    chars.append(secrets.choice(all_chars))

secrets.SystemRandom().shuffle(chars)
return "".join(chars)
```

- `all_chars` réunit les 4 catégories en une seule chaîne
- Il manque `length - len(chars)` caractères, soit 12 pour une longueur de 16
- `_` est le nom d'usage d'une variable de boucle dont on ne se sert pas
- Sans le mélange, le mot de passe commencerait **toujours** par une majuscule, une minuscule, un chiffre puis un spécial



Étape 4 – Tester avec les exemples

```
Tests

print(generate_password())      # par exemple : 45h#0_$APXE[QLje
print(generate_password(24))    # par exemple : {U/7/lzny-b{MJb[-BlagCX(

try:
    generate_password(8)
except ValueError as error:
    print("Erreur :", error)    # Erreur : longueur minimale : 12
```

Le `try` / `except` sert seulement à afficher l'erreur sans arrêter le programme de test.



Étape 4 – Vérifier 1 000 mots de passe

```
challenge3.py (suite)

invalid = 0
for _ in range(1000):
    valid, errors = check_password(generate_password())
    if not valid:
        invalid += 1
print("Mots de passe invalides sur 1000 :", invalid)
# Mots de passe invalides sur 1000 : 0
```

- `check_password()` renvoie un **tuple**, que l'on déballe en deux variables
- Un seul cas raté sur 1 000 suffirait à révéler un défaut. C'est l'intérêt de tester en grand nombre
- `challenge1.py` doit être **dans le même dossier** que `challenge3.py`

Les tests de `challenge1.py` s'affichent à l'import ? Placez-les sous `if __name__ == "__main__":`. Ils ne s'exécuteront plus que si l'on lance ce fichier directement.



Les pièges les plus fréquents

Piège	Correction
random au lieu de secrets	un secret doit être imprévisible. Utiliser secrets
16 caractères tirés parmi tous	rien ne garantit les 4 catégories. En tirer une de chaque d'abord
pas de mélange	les 4 premiers caractères suivent toujours le même ordre
<code>print("erreur")</code> au lieu de <code>raise</code>	la fonction continue. Lever une <code>ValueError</code>
<code>ImportError</code> sur <code>challenge1</code>	les deux fichiers doivent être dans le même dossier

Un bon programme **réutilise** ce qui existe : les modules de Python et son propre code, comme le défi 1.



Des questions ?

Chris CHEVALIER

chris.chevalier@ecole-hexagone.com